


# 为什么要并行计算？

王桢

wangzh665@mail.sysu.edu.cn

# 目录




中山大學
SUN YAT-SEN UNIVERSITY

- **Why we need ever-increasing performance.**
  - 大规模计算的价值!
- Why we're building parallel systems.
- Why we need to write parallel programs.
- How do we write parallel programs?
- What we'll be doing.
- Concurrent, parallel, distributed!

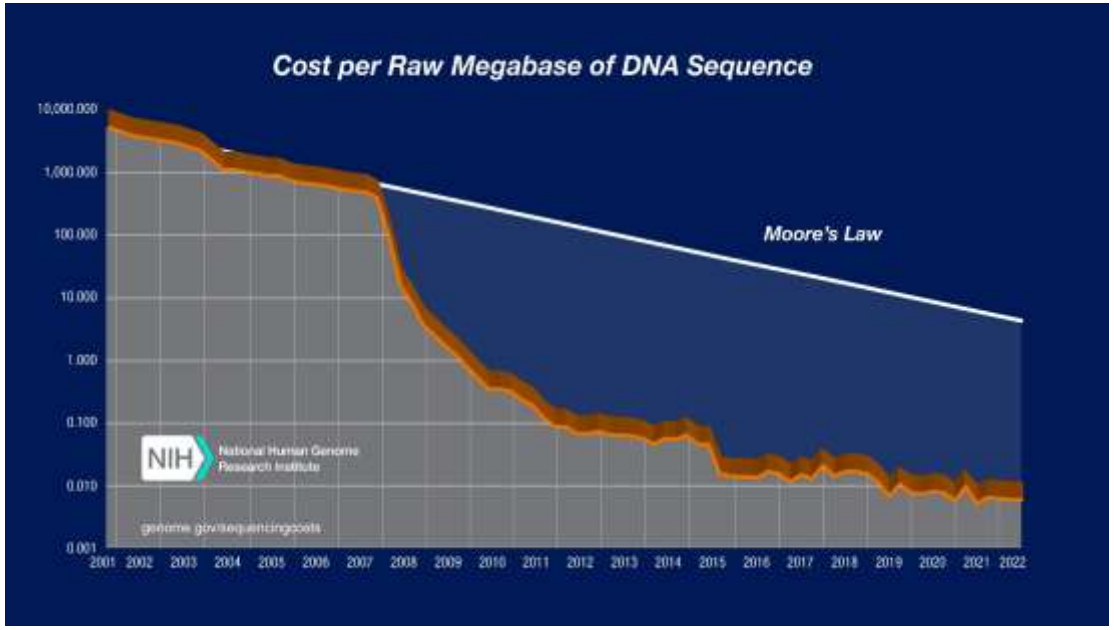
# 大规模计算的价值

中山大學
SUN YAT-SEN UNIVERSITY

- Computational power is increasing, but so are our computation problems and needs.
  - Big data acquired from nature through novel technologies or generated by society via the Internet.
  - Large models exhibit unprecedented performance that improves with increased model scale and computational budget.

- More complex problems are still waiting to be solved.

# 更大的数据




High-throughput genotyping




How much data is generated?

# 更大的模型

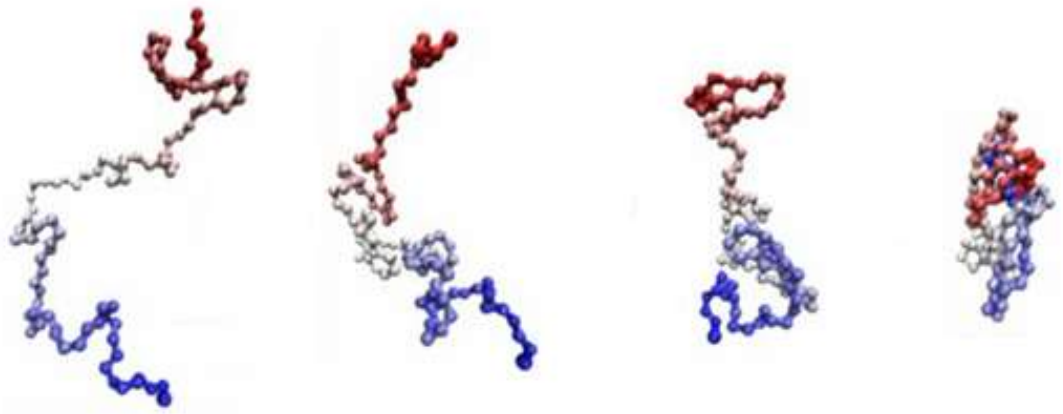


Climate modeling




LLMs

# 更多问题有待计算方法求解



Protein folding



Energy research

# 目录




- Why we need ever-increasing performance.
- **Why we're building parallel systems.**
  - 串行已经触及天花板
- Why we need to write parallel programs.
- How do we write parallel programs?
- What we'll be doing.
- Concurrent, parallel, distributed!

# 串行程序的性能




中山大學
SUN YAT-SEN UNIVERSITY

- Roughly, performance identity gives:
  - Execute time $= \frac{\text{Instructions}}{\text{Program}} \times \frac{\text{Cycles}}{\text{Instruction}} \times \frac{\text{Time}}{\text{Cycle}}$
  - Equivalently, $\text{Performance} \propto \frac{\text{Clock Rate}}{\text{CPI}}$

- How did clock rate and CPI scale in the past—and why did they stop improving?

# 处理器性能发展回顾



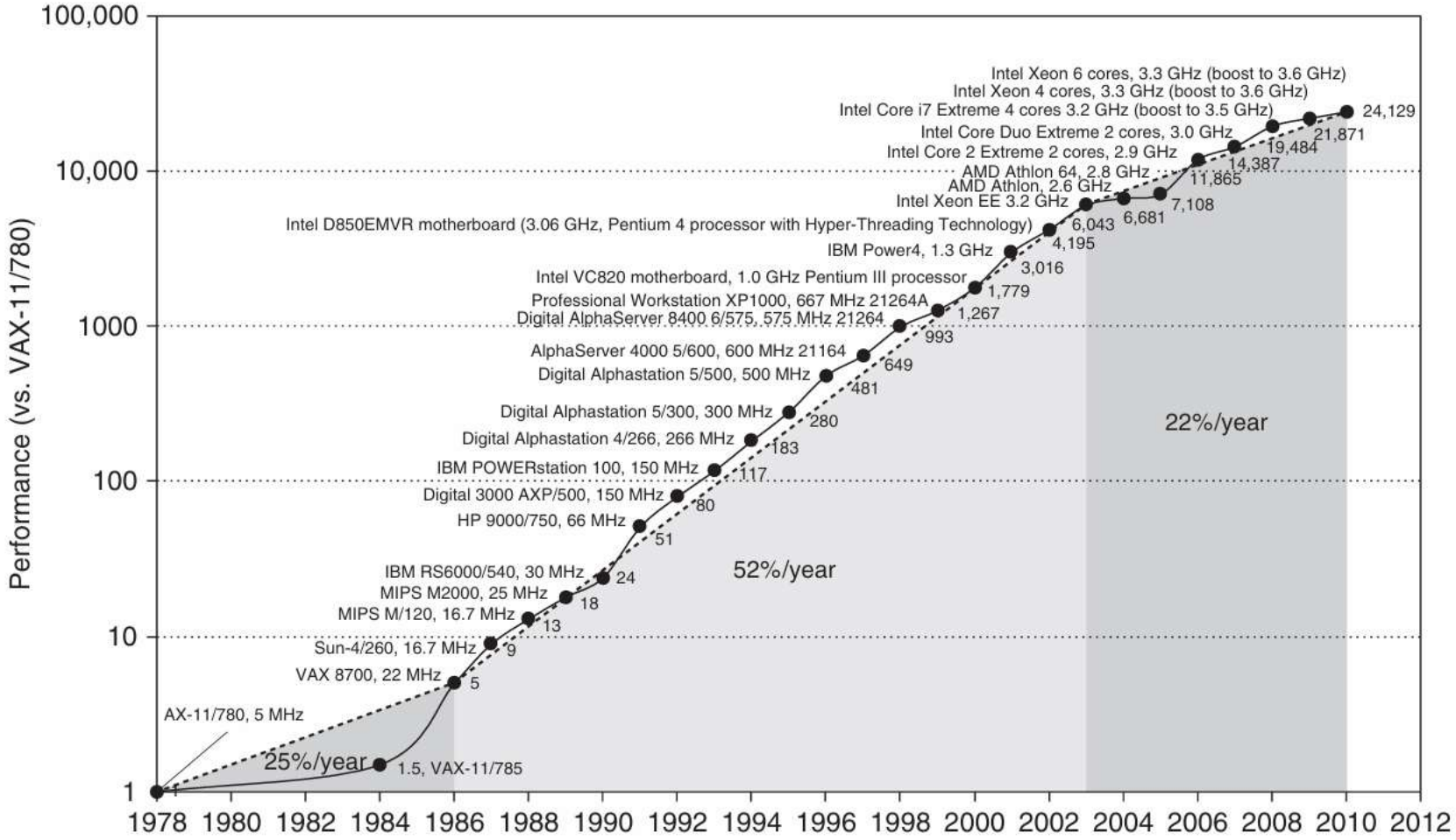
中山大學
SUN YAT-SEN UNIVERSITY

- Faster clock rates
  - e.g., 10 MHz → 200 MHz → 3 GHz
- Wider data paths
  - 4 bit → 8 bit → 16 bit → 32 bit → 64 bit
- More efficient pipelining
  - e.g., 3.5 Cycles Per Instruction (CPI) → 1.1 CPI
- Exploiting instruction-level parallelism (ILP)
  - “Superscalar”: e.g., issue up to 4 instructions/cycle
  - “Out-of-order”: extract parallelism from instruction stream

All these, among others, reduce CPI yet demand more transistors!

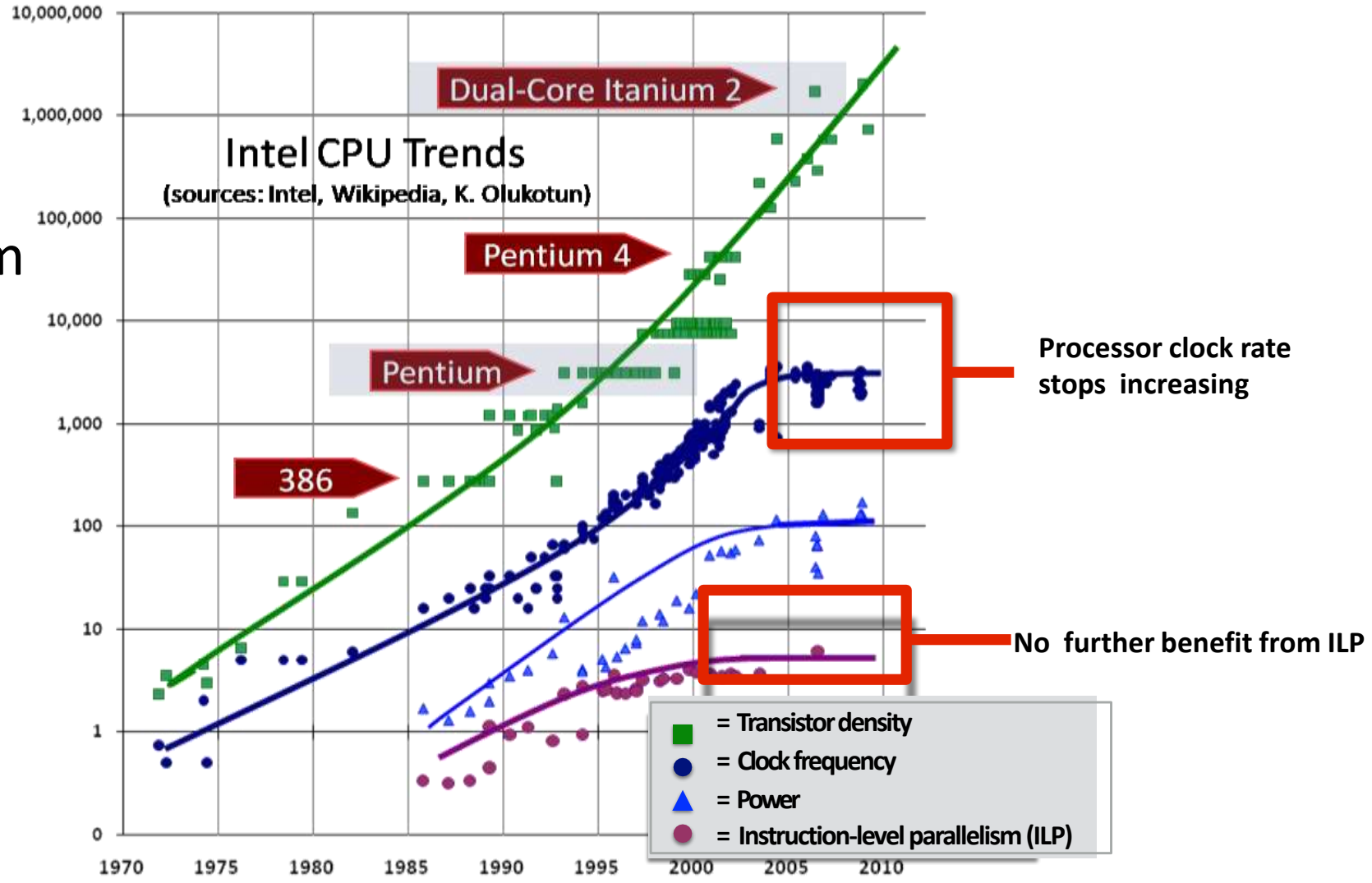
# 处理器性能发展回顾

- Slow down since 2023



# 处理器性能发展回顾

- End of frequency scaling
- And ILP hits program dependency wall
  - Most real programs expose only ~3–5 independent instructions

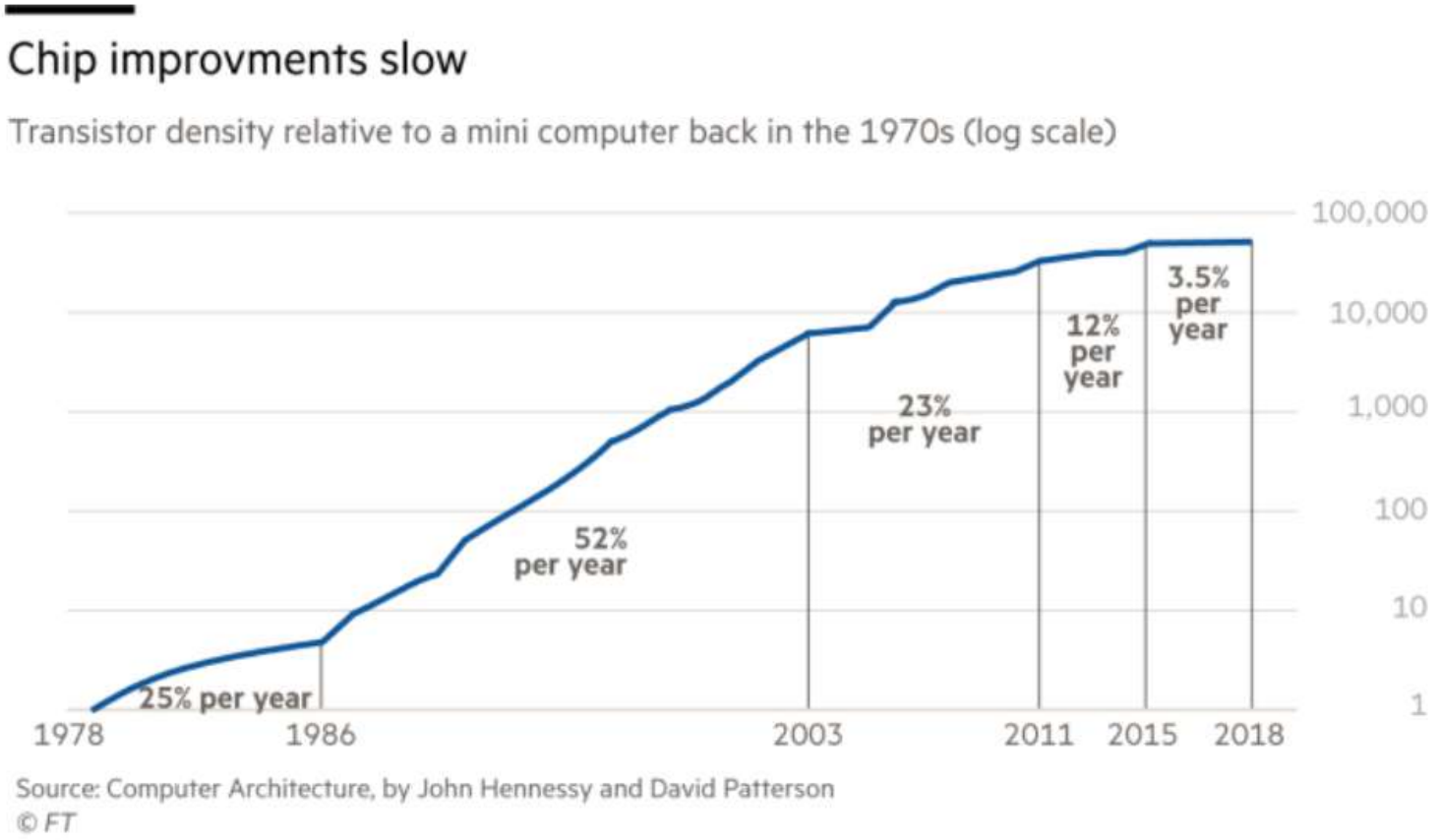


# 处理器性能发展回顾

- End of CPI decreasing due to the end of transistor density scaling

The number of transistors per chip would double every year --- by *Gordon Moore* in 1965.

Lasted for ~50 years but no longer holds.



# 为何难以继续提升晶体管密度?

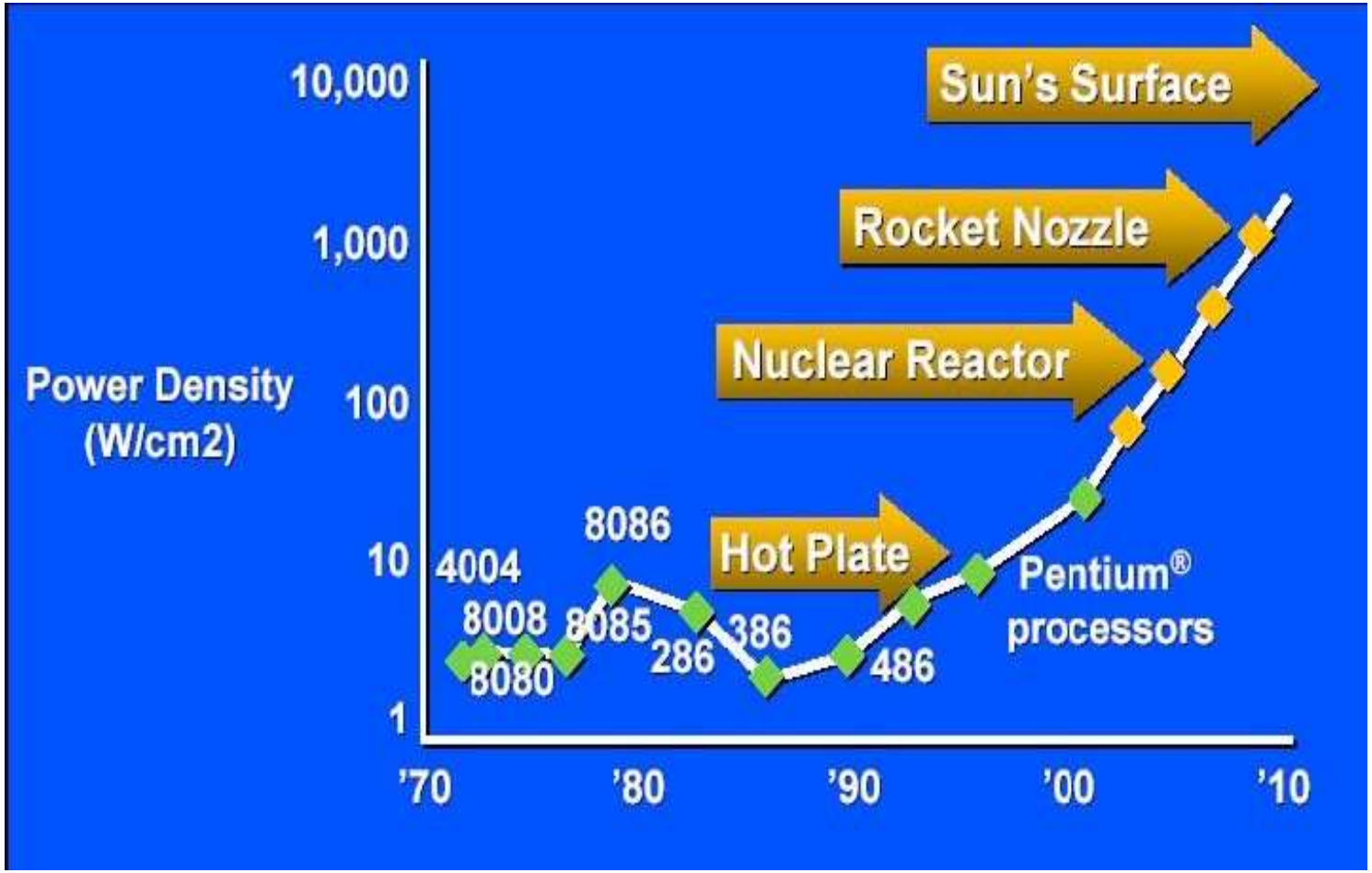



- 电子管 → 晶体管 → 集成电路 → 大规模集成电路
- Dennard scaling (by *Robert Dennard* in 1974):
  - "Power density was constant for a given area of silicon even as you increased the number of transistors because of smaller dimensions of each transistor."
  - higher transistor density → smaller CPI with constant power density ☺
- But ended ~2004
  - More transistors → increased power consumption → increased heat → unreliable processors.

# 为何难以继续提升晶体管密度?

中山大學 SUN YAT-SEN UNIVERSITY

- Intel hits the Power Density Wall



# 路在何方？并行系统！

- Intel publicly acknowledged that it had hit a "thermal wall" on its microprocessor line.





The New York Times

TECHNOLOGY

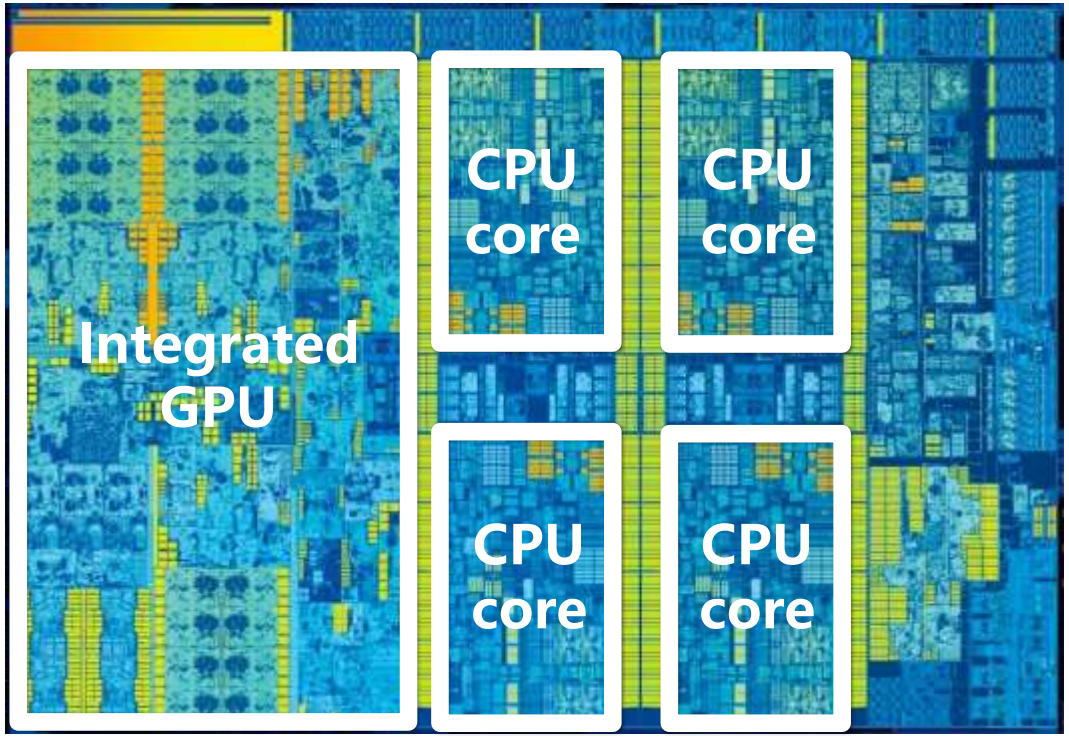
*TECHNOLOGY; Intel's Big Shift After Hitting Technical Wall*

By John Markoff

May 17, 2004

- Move away from single-core systems to **multicore processors.**
  - “core” = central processing unit (CPU)

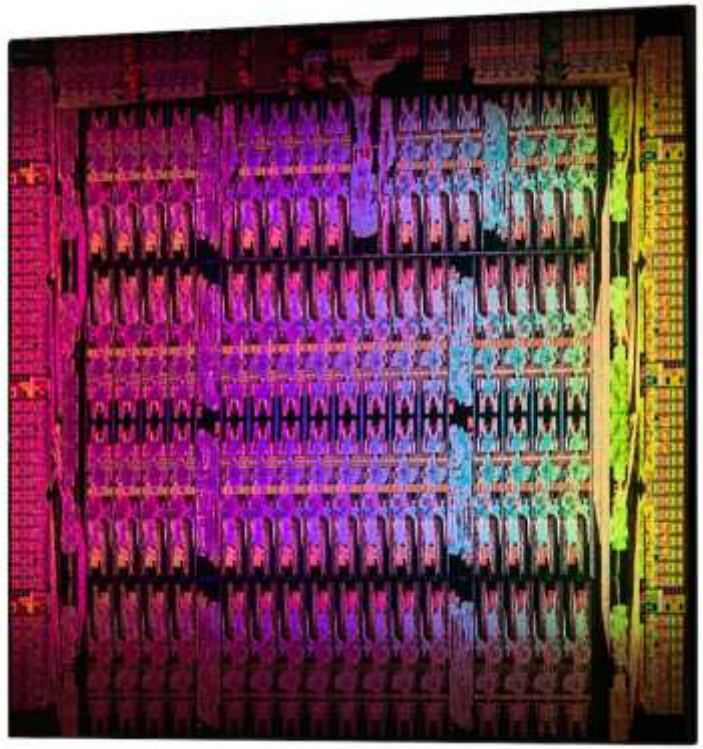
# 路在何方？并行系统！




Intel Skylake (2015) (aka “6th generation Core i7”)

Quad-core CPU + multi-core GPU integrated on one chip





Intel Xeon Phi 7120A “coprocessor”

61 “simple” x86 cores (1.3 Ghz)

Targeted as an accelerator for supercomputing applications

# 路在何方？并行系统！

中山大學 SUN YAT-SEN UNIVERSITY

Examples from Apple’s product line:



**Mac Pro**

**8 Intel Xeon E5 cores**



**iMac Pro**

**18 Intel Xeon W cores**



**MacBook Pro Retina 15” 6**

**Intel Core i9 cores**

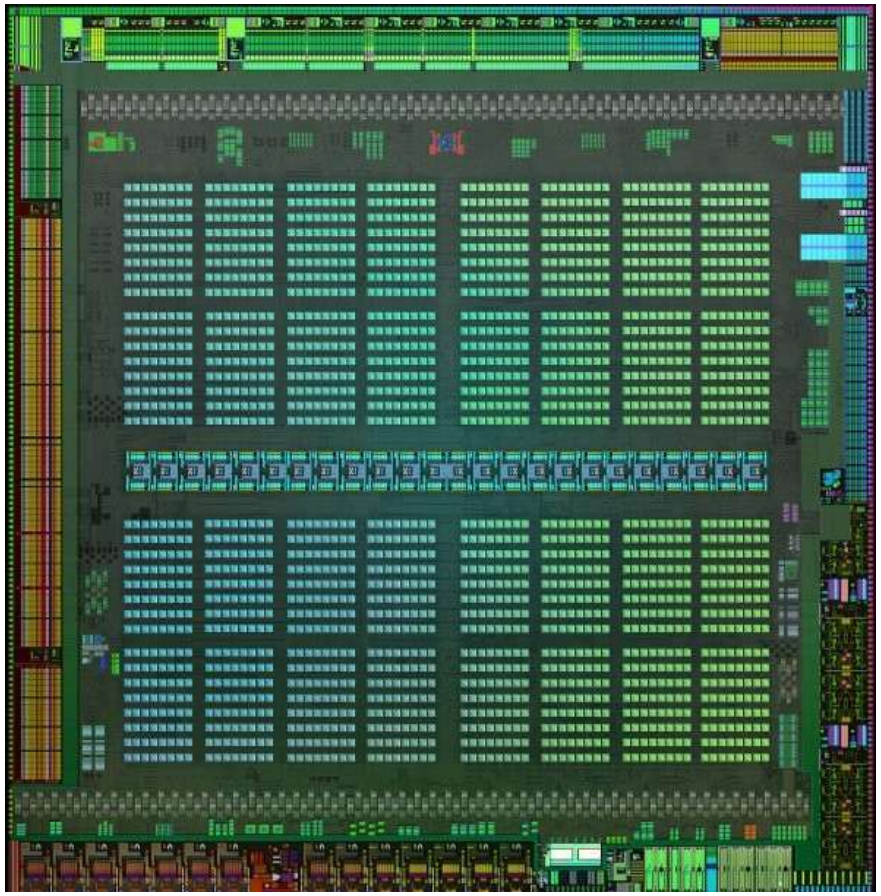


**iPhone XR**

**4 CPU cores**

**6 GPU cores**

# 路在何方？并行系统！



NVIDIA GV100 Volta GPU (2017)
80 major processing blocks
(but much, much more parallelism available... Will be detailed later in Chap6)

# 路在何方？并行系统！



中山大學
SUN YAT-SEN UNIVERSITY

- Supercomputing
  - Today: clusters of multi-core CPUs + GPUs
  - Oak Ridge National Lab: Summit
    - 4,608 nodes
    - Each with two 22-core CPUs + 6 GPUs



# 目录




中山大學
SUN YAT-SEN UNIVERSITY

- Why we need ever-increasing performance.
- Why we're building parallel systems.
- **Why we need to write parallel programs.**
  - 发挥并行系统的优势
- How do we write parallel programs?
- What we'll be doing.
- Concurrent, parallel, distributed!

# 如何提升程序性能?




- Answer before 2004:
  - Just wait 6 months, and buy a new machine!

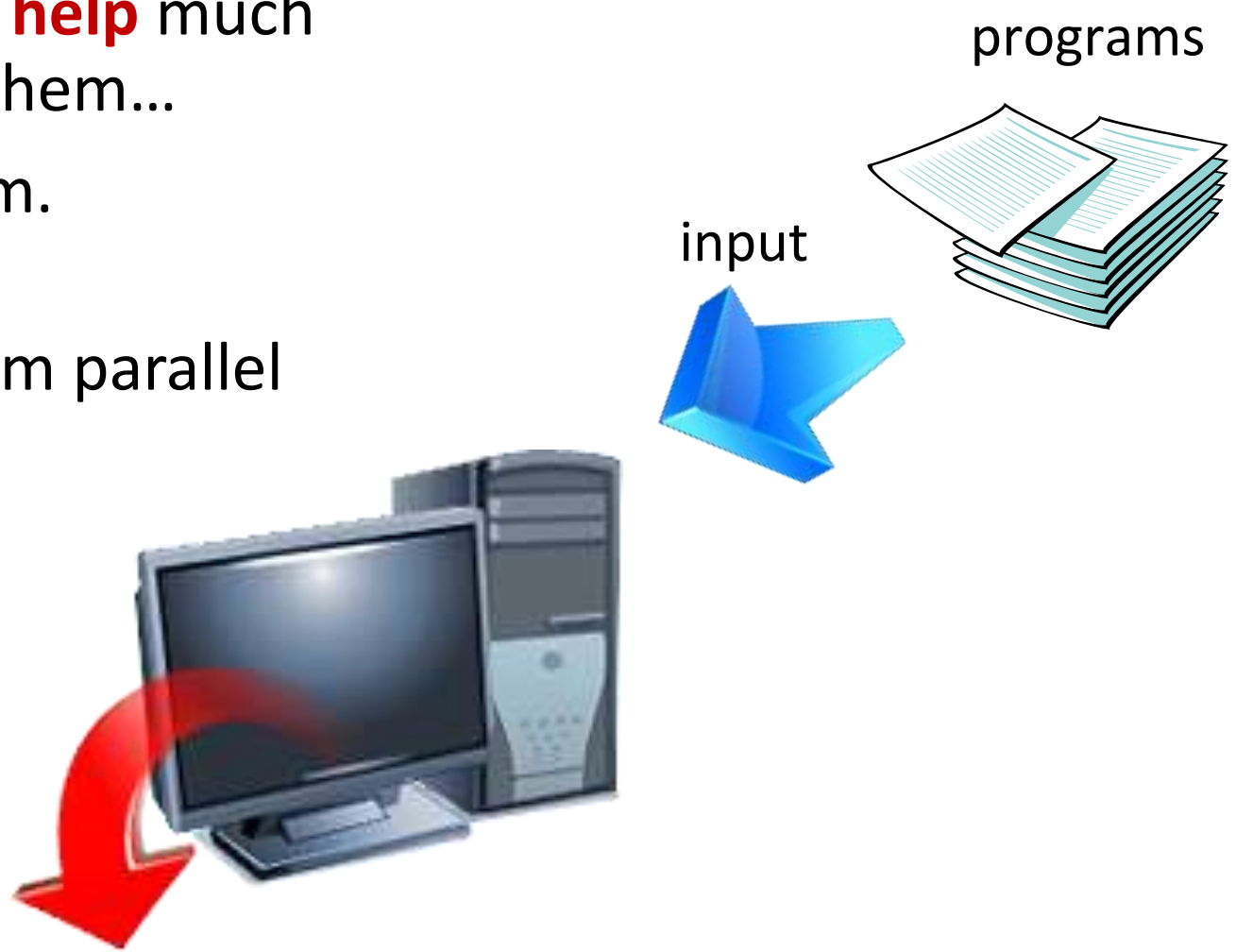
- Answer after 2004:
  - **You need to write parallel software.**

# 需要程序员开发并行程序

中山大學 SUN YAT-SEN UNIVERSITY

- Adding more processors **doesn't help** much if programmers aren't aware of them…
- … or don't know how to use them.

- Serial programs don't benefit from parallel system (in most cases).



# 需要程序员开发并行程序

中山大學
SUN YAT-SEN UNIVERSITY

- Running multiple instances of a serial program often isn't very useful.
- Think of running multiple instances of your favorite game.
- What you really want is for it to **run faster**.



# 将串行程序翻译为并行程序?




中山大學
SUN YAT-SEN UNIVERSITY

- Rewrite serial programs so that they're parallel.
- Write translation programs that automatically convert serial programs into parallel programs.
  - This is **very difficult** to do.
  - Success has been limited.

# 将串行程序翻译为并行程序?




- Some coding constructs can be recognized by an automatic program generator, and converted to a parallel construct.
- However, it's likely that the result will be a very inefficient program.
- Sometimes the best parallel solution is to step back and devise an entirely new algorithm.

# Example

- Compute $n$ values and add them together.

- Serial solution:

```
sum = 0;
for (i = 0; i < n; i++) {
   x = Compute_next_value(. . .);
   sum += x;
}
```

# Example (cont.)



中山大學
SUN YAT-SEN UNIVERSITY

- We have $p$ cores, $p$ much smaller than $n$.
- Each core performs a partial sum of approximately $\frac{n}{p}$ values.

```
my_sum = 0;
my_first_i = . . . ;
my_last_i = . . . ;
for (my_i = my_first_i; my_i < my_last_i; my_i++) {
   my_x = Compute_next_value( . . .);
   my_sum += my_x;
}
```

Each core uses it's own private variables and executes this block of code independently of the other cores.

# Example (cont.)




中山大學
SUN YAT-SEN UNIVERSITY

- After each core completes execution of the code, a private variable `my_sum` contains the sum of the values computed by its calls to `Compute_next_value`.

- Ex., $p = 8, n = 24$, then the calls to `Compute_next_value` return:

1,4,3, 9,2,8, 5,1,1, 5,2,7, 2,5,0, 4,1,8, 6,5,1, 2,3,9

# Example (cont.)

- Once all the cores are done computing their private `my_sum`, they form a global sum by sending results to a designated "master" core which adds the final result.

```
if (I'm the master core) {
   sum = my_x;
   for each core other than myself {
      receive value from core;
      sum += value;
   }
} else {
   send my_x to the master;
}
```

# Example (cont.)

| Core | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| my_sum | 8 | 19 | 7 | 15 | 7 | 13 | 12 | 14 |



Global sum

8 + 19 + 7 + 15 + 7 + 13 + 12 + 14 = 95

| Core | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 |
|---|---|---|---|---|---|---|---|---|
| my_sum | 95 | 19 | 7 | 15 | 7 | 13 | 12 | 14 |

## But wait!

There's a much better way to compute the global sum.



# Better parallel algorithm




中山大學
SUN YAT-SEN UNIVERSITY

- Don’t make the master core do all the work.
- Share it among the other cores.

- Pair the cores so that core 0 adds its result with core 1’s result.
- Core 2 adds its result with core 3’s result, etc.
- Work with odd and even numbered pairs of cores.

# Better parallel algorithm

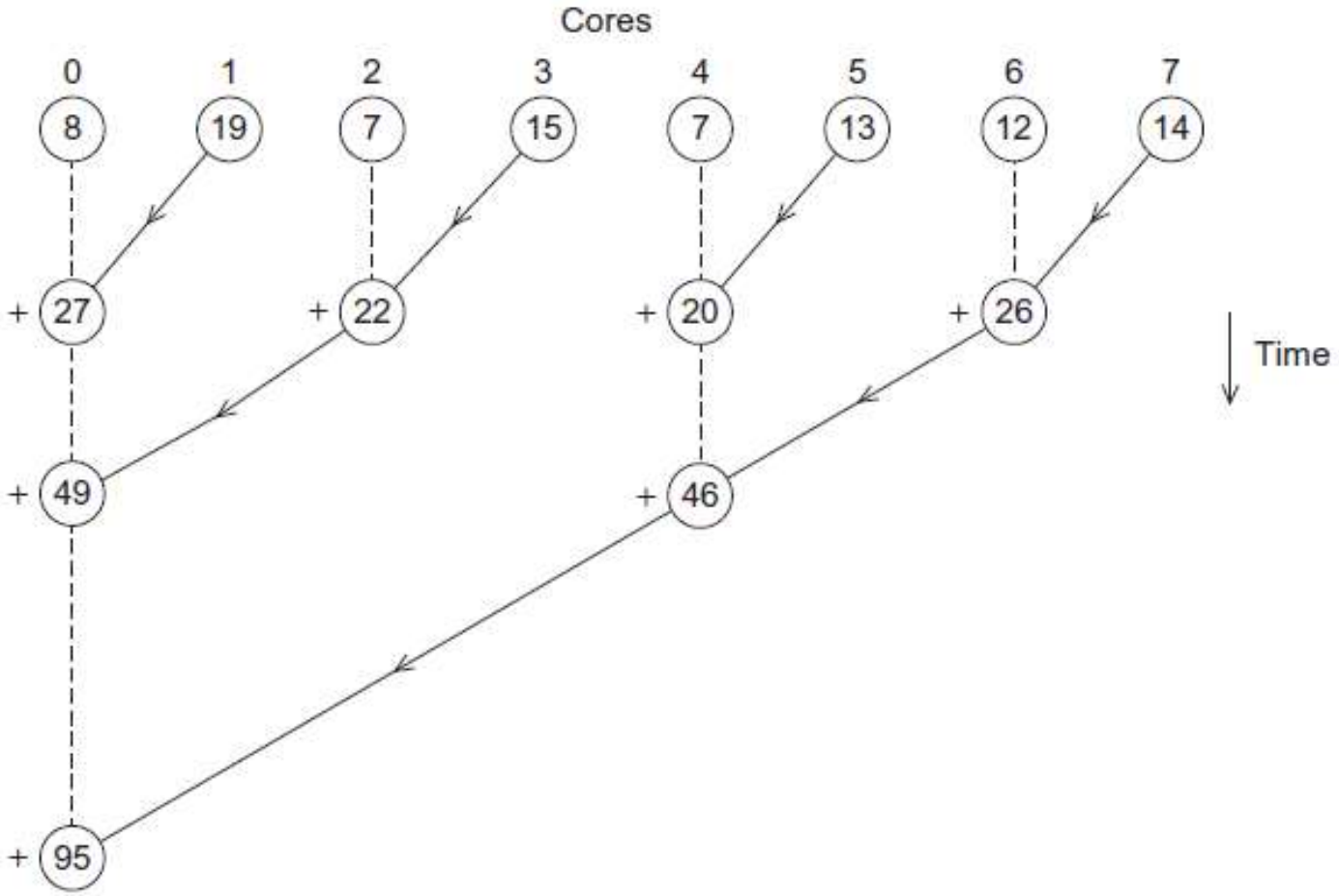



中山大學
SUN YAT-SEN UNIVERSITY

- Repeat the process now with only the evenly ranked cores.
- Core 0 adds result from core 2.
- Core 4 adds the result from core 6, etc.

- Now cores divisible by 4 repeat the process, and so forth, until core 0 has the final result.

# Better parallel algorithm



# Better parallel algorithm (Analysis)




中山大學
SUN YAT-SEN UNIVERSITY

- In the first example, the master core performs 7 receives and 7 additions.
- In the second example, the master core performs 3 receives and 3 additions.
- The improvement is more than a factor of 2!

# Better parallel algorithm (Analysis)



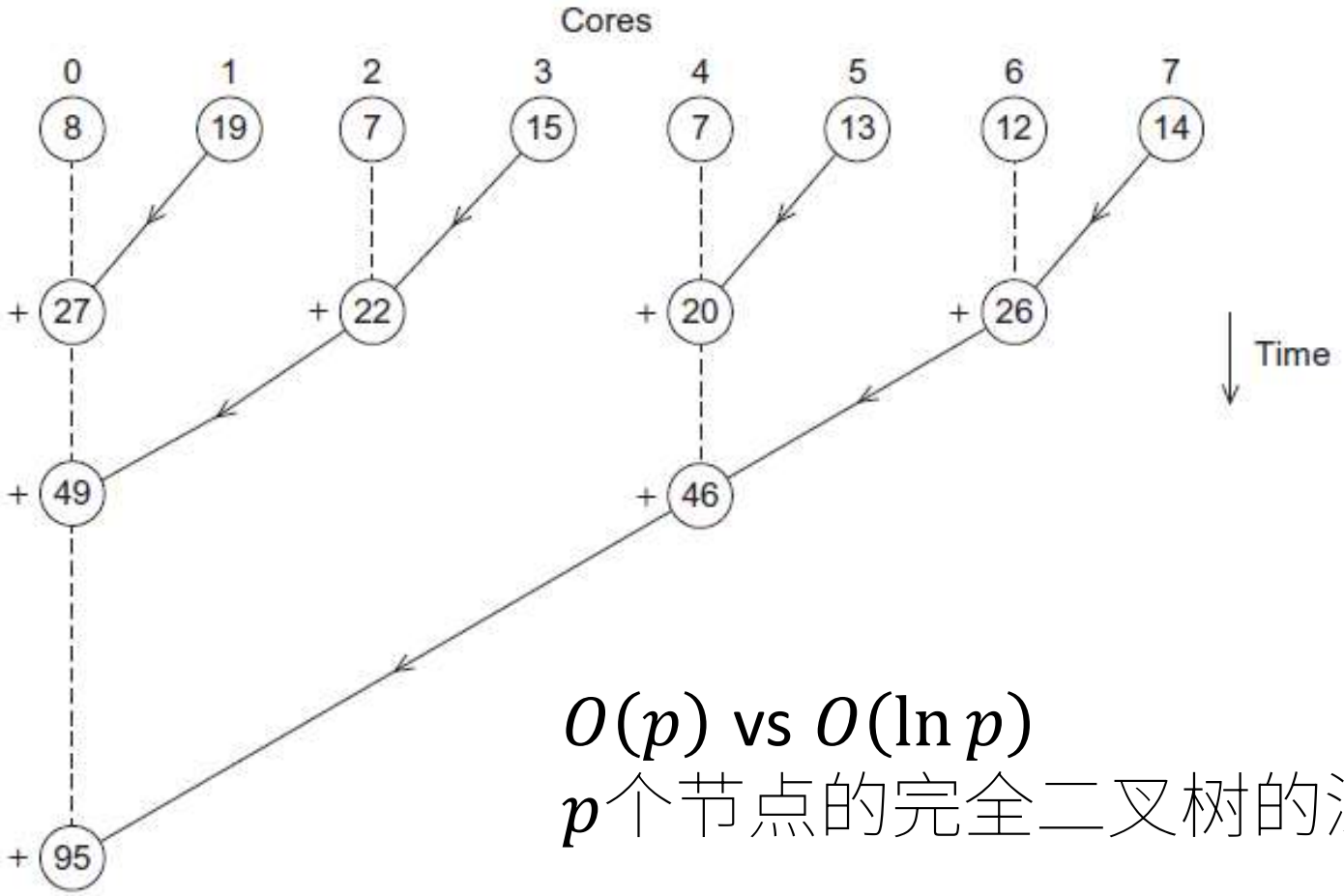

中山大學
SUN YAT-SEN UNIVERSITY

- The difference is more dramatic with a larger number of cores.

- If we have 1000 cores:
  - The first example would require the master to perform 999 receives and 999 additions.
  - The second example would only require 10 receives and 10 additions.

- That's an improvement of almost a factor of 100!

# Better parallel algorithm (Analysis)




$O(p)$ vs $O(\ln p)$

$p$个节点的完全二叉树的深度$O(\ln p)$

# 目录

- Why we need ever-increasing performance.
- Why we’ re building parallel systems.
- Why we need to write parallel programs.
- **How do we write parallel programs?**
  - **并行的基本类型**
- What we’ ll be doing.
- Concurrent, parallel, distributed!

# 并行的基本类型




中山大學
SUN YAT-SEN UNIVERSITY

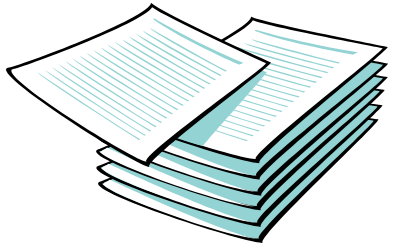
- Task parallelism
  - Partition various tasks carried out solving the problem among the cores.
- Data parallelism
  - Partition the data used in solving the problem among the cores.
  - Each core carries out similar operations on it’s part of the data.

# Example



15 questions

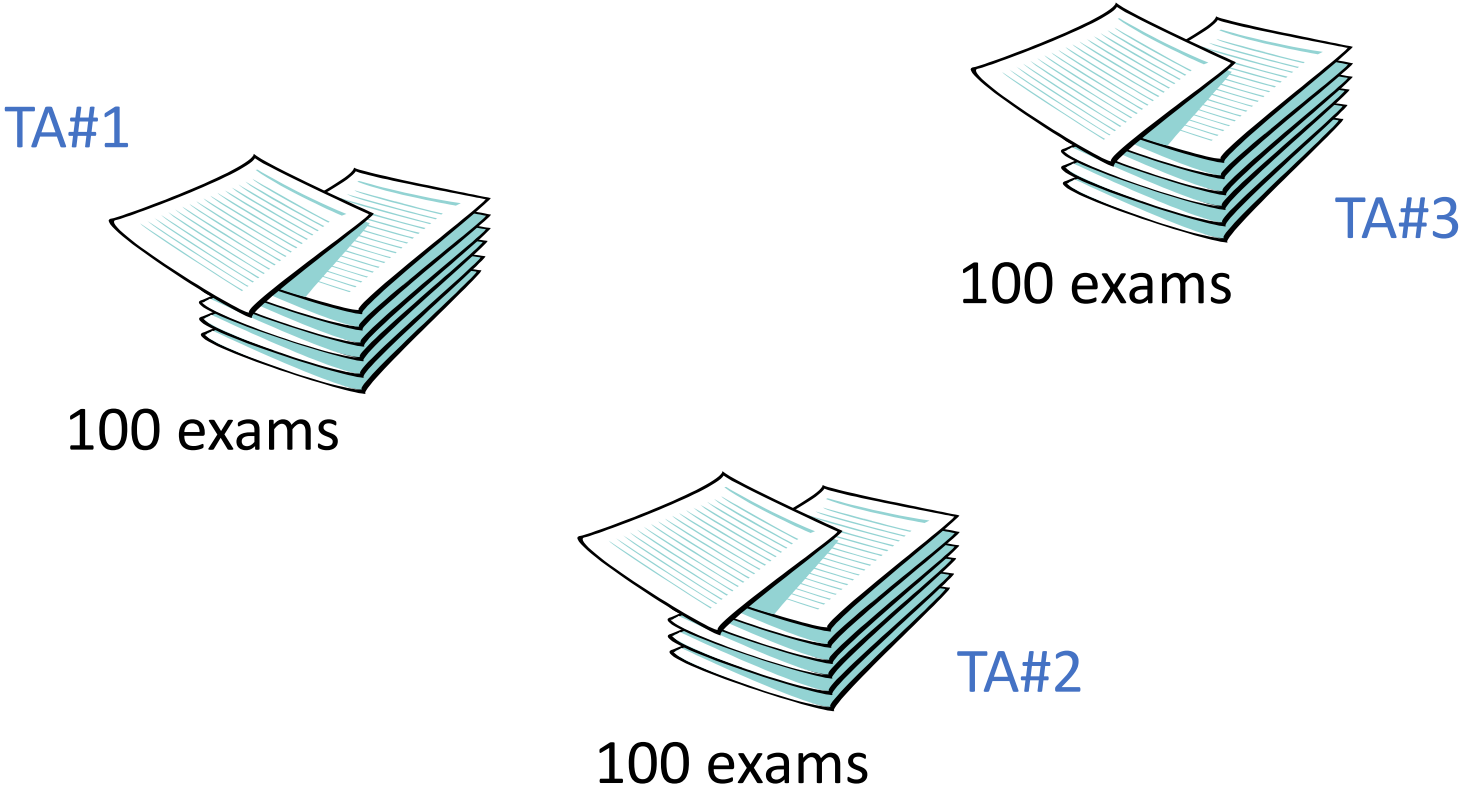
300 exams






Professor’s grading assistants

# 并行的基本类型：数据并行



# 并行的基本类型：任务并行



TA#1

Questions 1 - 5



TA#3

Questions 11 - 15



TA#2

Questions 6 - 10

# 并行的基本类型：数据并行



中山大學
SUN YAT-SEN UNIVERSITY

```
sum = 0;
for (i = 0; i < n; i++) {
    x = Compute_next_value(. . .);
    sum += x;
}
```

# 并行的基本类型：任务并行



中山大學
SUN YAT-SEN UNIVERSITY

```
if (I'm the master core) {
    sum = my_x;
    for each core other than myself {
        receive value from core;
        sum += value;
    }
} else {
    send my_x to the master;
}
```

Tasks

1) Sending
2) Receiving+Addition

# Coordination




中山大學
SUN YAT-SEN UNIVERSITY

- Cores usually need to **coordinate** their work.

- Communication – one or more cores send their current partial sums to another core.
- Load balancing – share the work evenly among the cores so that one is not heavily loaded.
- Synchronization – because each core works at its own pace, make sure cores do not get too far ahead of the rest.

# 目录




- Why we need ever-increasing performance.
- Why we're building parallel systems.
- Why we need to write parallel programs.
- How do we write parallel programs?
- **What we'll be doing.**
  - 课程涉及的并行框架
- Concurrent, parallel, distributed!

# What we'll be doing




中山大學
SUN YAT-SEN UNIVERSITY

- Learning to write programs that are explicitly parallel.
- Using the C and CPP language.
- Using three different extensions to C/CPP.
  - Message-Passing Interface (MPI) for distributed-memory system
  - OpenMP for shared-memory system
  - Posix Threads (Pthreads) for shared-memory system
  - CUDA for hybrid system

# Type of parallel systems

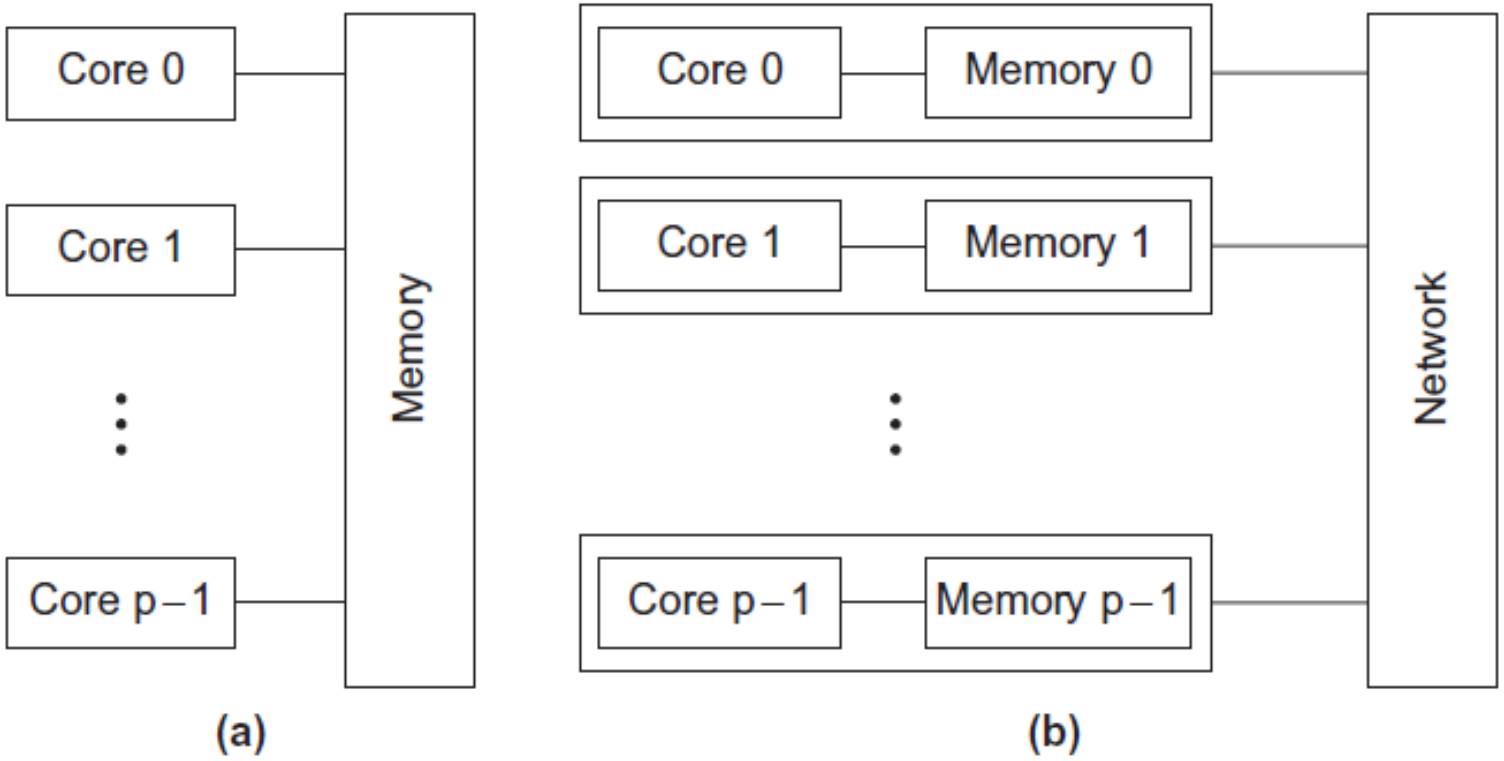



中山大學
SUN YAT-SEN UNIVERSITY

- Shared-memory
  - The cores can share access to the computer's memory.
  - Coordinate the cores by having them examine and update shared memory locations.

- Distributed-memory
  - Each core has its own, private memory.
  - The cores must communicate explicitly by sending messages across a network.

# Type of parallel systems




Shared-memory

Distributed-memory

# 目录



中山大學
SUN YAT-SEN UNIVERSITY

- Why we need ever-increasing performance.
- Why we’ re building parallel systems.
- Why we need to write parallel programs.
- How do we write parallel programs?
- What we’ ll be doing.
- **Concurrent, parallel, distributed!**
  - **关键术语的含义**

# Terminology




中山大學
SUN YAT-SEN UNIVERSITY

- Concurrent computing – a program is one in which multiple tasks can be in progress at any instant.

- Parallel computing – a program is one in which multiple tasks cooperate closely to solve a problem

- Distributed computing – a program may need to cooperate with other programs to solve a problem.

# 小结（1）




- The laws of physics have brought us to the doorstep of multicore technology.
- Serial programs typically don’t benefit from multiple cores.
- Automatic parallel program generation from serial program code isn’t the most efficient approach to get high performance from multicore computers.

# 小结（2）




- Learning to write parallel programs involves learning how to coordinate the cores.

- Parallel programs are usually very complex and therefore, require sound program techniques and development.

# 谢谢！

- Q&A

中山大學
SUN YAT-SEN UNIVERSITY